

DOCUMENTACION: RECETARIO DIGITAL

Alan González Quintero

Estructuras de Datos

Índice

REQUERIMIENTOS.....	4
1. Gestión de Recetas	4
2. Gestión de Ingredientes	5
3. Persistencia.....	5
4. Estructura de Datos a Utilizar	5
DISEÑO DEL SISTEMA	6
Arquitectura General	6
UML del sistema.....	6
Diagramas.....	7
1. Diagrama de clases	7
2. Diagramas de secuencia	8
Seudocódigo (resumen del flujo principal)	11
CODIFICACION	12
Traducción del diseño al código	12
Código fuente completo	12
<i>Include/</i>	12
<i>src/</i>	26
PRUEBAS.....	46
1. Listar Recetas	46
2. Listar por categoría	47
3. Buscar receta	49
4. Eliminar receta	50
5. Eliminar todas las recetas	51
6. Agregar receta	51
7. Ordenar por nombre	52
8. Ordenar por tiempo de preparación	53
9. Agregar ingrediente (ordenado)	53
10. Eliminar ingrediente	55
11. Modificar cantidad de ingrediente	56
12. Modificar procedimiento	56
13. Guardar a disco	57
14. Leer de disco	58
PUESTA EN MARCHA.....	60
Manual de Instalación	60
Requerimientos de hardware y software	60
Estructura de archivos del proyecto	60
Compilación en línea de comandos (g++)	61
Ubicación de archivos de datos	61
Manual de operación	62

<i>Inicio del programa</i>	62
<i>Flujo general de uso</i>	62
<i>Mensajes y manejo de errores</i>	64

REQUERIMIENTOS

El objetivo del proyecto es desarrollar un sistema de recetario digital usando el paradigma de Programación Orientada a Objetos y Estructuras de Datos.

El programa debe permitir registrar, consultar, modificar y eliminar recetas, así como gestionar los ingredientes asociados a cada una.

Los requerimientos funcionales del sistema son los siguientes:

1. Gestión de Recetas

El sistema debe permitir:

- 1.1) Agregar una nueva receta con:
 - Nombre del platillo
 - Autor
 - Categoría (Desayuno, Almuerzo, Cena o Navideño)
 - Tiempo de preparación
 - Procedimiento
 - Lista de ingredientes (ordenada desde la inserción)
- 1.2) Mostrar todas las recetas existentes, mostrando:
 - Todos los datos del punto 1.1
 - Lista completa de ingredientes
- 1.3) Buscar recetas por:
 - Nombre exacto
 - Categoría (mostrar la primera encontrada)
- 1.4) Modificar una receta:
 - Cambiar procedimiento
 - Agregar ingredientes
 - Eliminar ingredientes
 - Modificar la cantidad de un ingrediente
- 1.5) Eliminar:
 - Una receta por nombre
 - Todas las recetas del recetario

2. Gestión de Ingredientes

Cada receta debe contener una lista simplemente ligada ordenada, donde se permita:

- Insertar un ingrediente manteniendo el orden alfabético
- Eliminar un ingrediente por nombre
- Eliminar todos los ingredientes
- Actualizar la cantidad de un ingrediente
- Consultar la lista completa

3. Persistencia

El programa debe permitir:

- Guardar el recetario completo en un archivo de texto.
- Cargar un recetario desde archivo
- Mantener el formato exacto entre sesiones

4. Estructura de Datos a Utilizar

La versión final deberá implementarse mediante:

- Lista doblemente ligada genérica (template) para las recetas
- Lista simplemente ligada genérica (template) para los ingredientes
- Ordenamiento recursivo sobre la lista doblemente ligada
- Comparadores explícitos tipo lambda (punto extra

DISEÑO DEL SISTEMA

Arquitectura General

El programa se divide en los siguientes módulos:

- Recipe – Representa una receta completa
- Ingredient – Almacena nombre, cantidad y unidad
- RecipeList – Lista doblemente ligada de recetas
- SinglyLinkedList – Lista simplemente ligada de ingredientes
- DoublyLinkedList – Implementación genérica de lista doble
- Menu – Interfaz de Usuario
- Storage – Guardar / Leer desde archivo

UML del sistema

Incluye:

1. Diagrama de clases
2. Diagramas de secuencia
 - 2.1) Agregar Receta con ingredientes
 - 2.2) Inserción ordenada de ingredientes
 - 2.3) Ordenar recetas por nombre
3. Diagrama de flujo/UI del menú

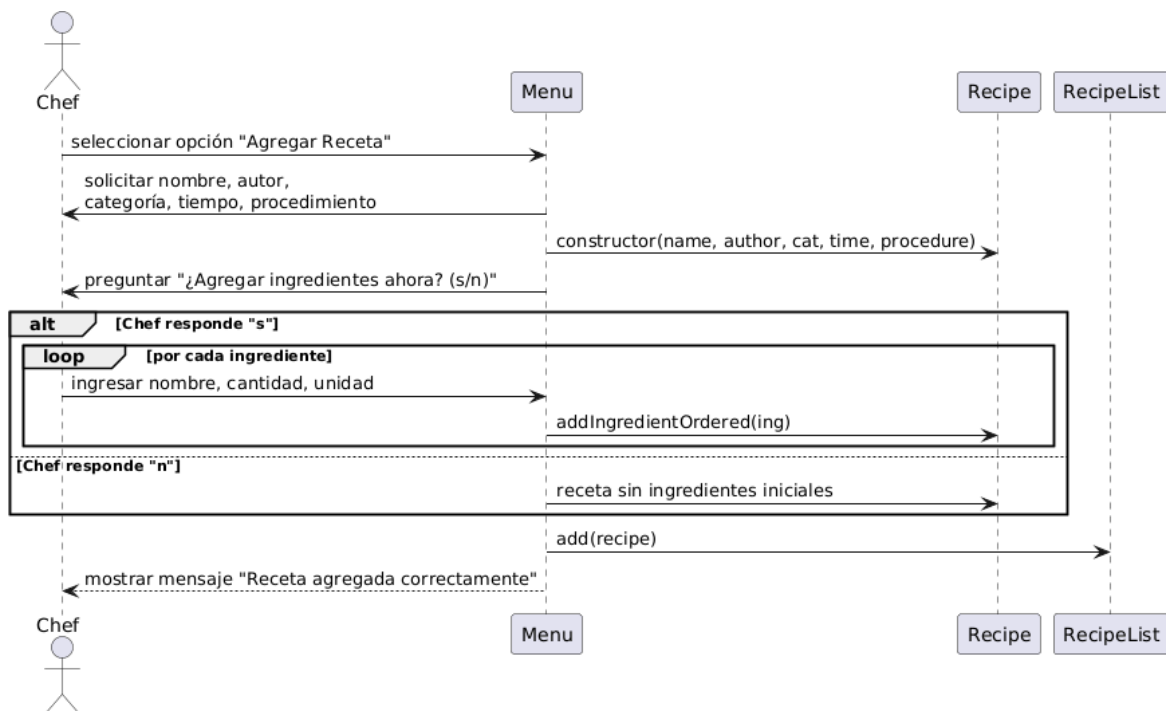
Diagramas

1. Diagrama de clases

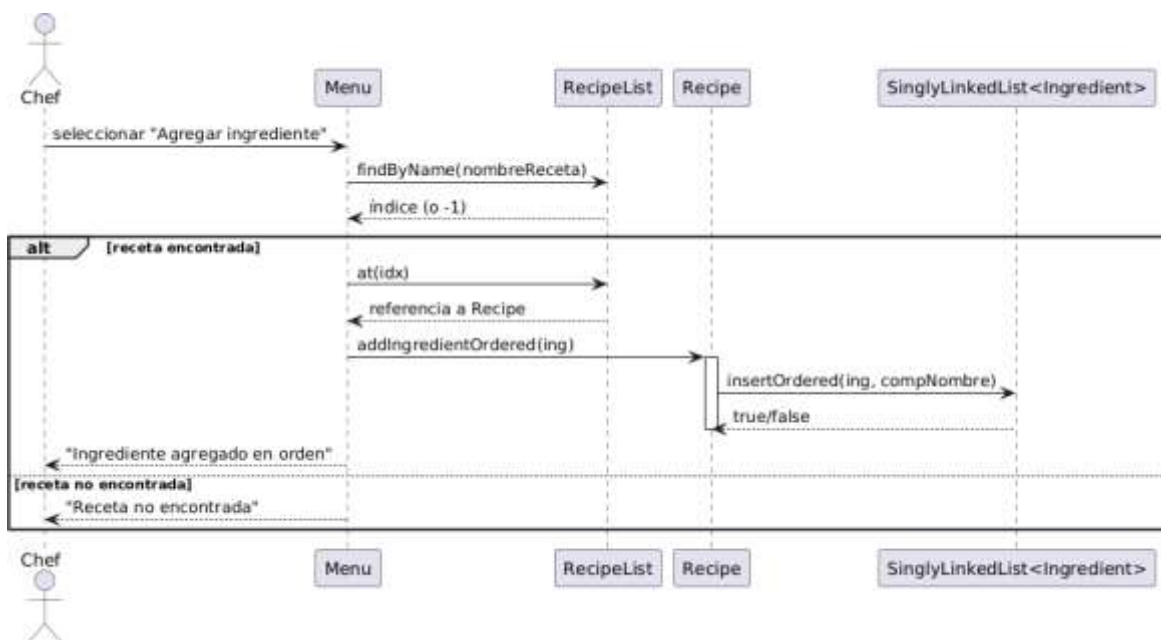


2. Diagramas de secuencia

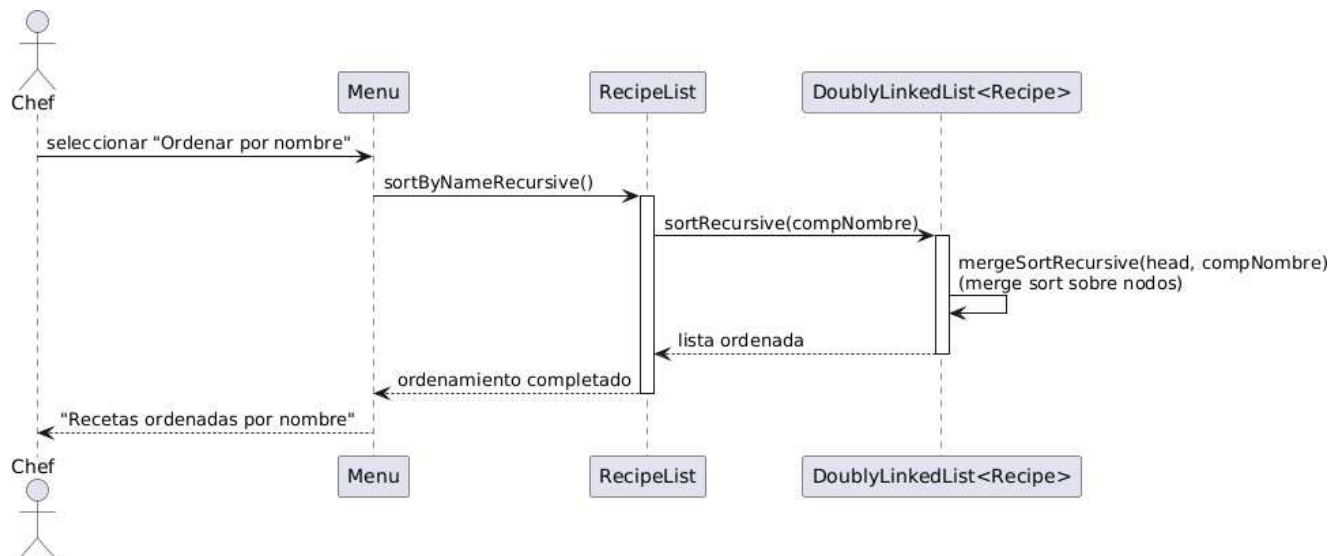
2.1 Agregar Receta con Ingredientes



2.2 Inserción ordenada de ingredientes



2.3 Ordenar recetas por nombre



Seudocódigo (resumen del flujo principal)

INICIO

cargar configuración UTF-8

inicializar lista de recetas

MIENTRAS (opción !=

salir) mostrar menú

leer opción

SEGÚN

opción

1: listar recetas

2: listar por categoría

3: buscar receta

4: agregar receta

5: eliminar receta

6: eliminar todas

7: ordenar por nombre

8: ordenar por tiempo

9: agregar ingrediente

10: eliminar ingrediente

11: modificar cantidad

12: modificar procedimiento

13: guardar en disco

14: leer desde disco

FIN SEGÚN

FIN MIENTRAS

FIN

CODIFICACION

Traducción del diseño al código

El sistema se implementará en C++ 17 siguiendo fielmente el diseño UML.

- Usando plantillas para generalizar las estructuras listas.
- Las recetas se almacenan en una lista doblemente ligada.
- Los ingredientes se almacenan en una lista simplemente ligada con inserción ordenada.
- El ordenamiento que se implementará será merge sort recursivo sobre nodos, usando lambdas de comparación.
- Las clases separadas en include/ y src/ , siguiendo buenas prácticas de modularidad.

Código fuente completo

Include/

category.hpp

```
#ifndef CATEGORY_HPP
#define CATEGORY_HPP

#include <string>

//Enum para categorias enum
class Category { Breakfast = 0,
    Lunch      = 1,
    Dinner     = 2,
    Christmas = 3
};

//Convierte Category a string
inline std::string categoryToString(Category c) { switch (c) {
    case Category::Breakfast: return "Desayuno"; case
    Category::Lunch:         return "Almuerzo";
    case Category::Dinner:   return "Cena"; case
    Category::Christmas:    return "Navideño";
    }
    return "Desayuno";
}
```

```

inline Category categoryFromString(const std::string& s) { if (s ==
    "Desayuno") return Category::Breakfast;
    if (s == "Almuerzo") return Category::Lunch; if (s ==
    "Cena") return Category::Dinner;
    if (s == "Navideño") return Category::Christmas; return
    Category::Breakfast;
}

#endif // CATEGORY_HPP

```

constants.hpp

```

#ifndef CONSTANTS_HPP
#define CONSTANTS_HPP

//Constantes que se usan en el proyecto constexpr int
MAX_RECIPES = 200;
constexpr int MAX_INGREDIENTS = 60;          //ingredientes por receta #endif

// CONSTANTS_HPP

```

Ingredient.hpp

```

#ifndef INGREDIENT_HPP
#define INGREDIENT_HPP

#include <string> #include
<sstream>

//Clase simple de Ingrediente class
Ingredient {
private:
    std::string name; double
    quantity {0.0}; std::string
    unit;

public:
    //Constructores Ingredient() =
    default;
    Ingredient(const std::string& name, double quantity, const std::string& unit)
        : name(name), quantity(quantity), unit(unit) {}

```

```

//Getters
const std::string& getName() const { return name; } double
getQuantity() const { return quantity; } const std::string&
getUnit() const { return unit; }

//Setters
void setQuantity(double q) { quantity = q; }

std::string toString() const { std::ostringstream oss;
    oss << name << " : " << quantity << " " << unit; return oss.str();
}
};

#endif // INGREDIENT_HPP

```

recipe.hpp

```

#ifndef RECIPE_HPP
#define RECIPE_HPP

#include <string> #include
<sstream> #include
"ingredient.hpp" #include
"category.hpp" #include
"constants.hpp"
#include "singly_linked_list.hpp"

class Recipe {
private:
    std::string name; std::string
    authorName;
    Category category {Category::Breakfast}; int
    prepTimeMinutes {0};
    std::string procedure; SinglyLinkedList<Ingredient>

    ingredients;

public:
    //Constructores Recipe() =
    default;
    Recipe(const std::string& name,

```

```

        const std::string& authorName, Category
        category,
        int prepTimeMinutes,
        const std::string& procedure)
: name(name), authorName(authorName),
  category(category),
  prepTimeMinutes(preptimeMinutes),
  procedure(procedure) {}

//Getters
const std::string& getName() const { return name; }
const std::string& getAuthorName() const { return authorName; } Category
getCategory() const { return category; }
int getPrepTimeMinutes() const { return prepTimeMinutes; } const
std::string& getProcedure() const { return procedure; }

//Setters
void setProcedure(const std::string& text) { procedure = text; }

//Acceso de solo lectura a ingredientes int
getIngredientCount() const { return
static_cast<int>(ingredients.size()); }
const Ingredient& getIngredientAt(int idx) const { return ingredients.at(idx); }

//Inserción ORDENADA por nombre
bool addIngredientOrdered(const Ingredient& ing);

//Eliminar ingrediente por nombre
bool removeIngredientByName(const std::string& ingName);

//Eliminar todos los ingredientes void
clearIngredients();

//Modificar cantidad de ingrediente
bool updateIngredientQuantity(const std::string& ingName, double newQty);

//Imprimir receta completa con ingredientes void
printDetails() const;

//toString básico
std::string toString() const { std::ostringstream oss;

```

```

        oss << "Receta{nombre=" << name
            << ", autor=" << authorName
            << ", categoria=" << categoryToString(category)
            << ", preparacion=" << prepTimeMinutes << " min}"; return
        oss.str();
    }
};

#endif // RECIPE_HPP

```

storage.hpp

```

#ifndef STORAGE_HPP
#define STORAGE_HPP

#include <string>
#include "recipe_list.hpp"

class Storage {
public:
    static bool saveToFile(const RecipeList& list, const std::string& path); static bool
    loadFromFile(RecipeList& list, const std::string& path);
};

#endif // STORAGE_HPP

```

recipe_list.hpp

```

#ifndef RECIPE_LIST_HPP
#define RECIPE_LIST_HPP

#include <string> #include
<iostream> #include
"recipe.hpp" #include
"constants.hpp"
#include "doubly_linked_list.hpp"

//Lista de recetas
class RecipeList : public DoublyLinkedList<Recipe> { public:
    bool add(const Recipe& recipe) { pushBack(recipe);
        return true;
    }
};

```



```

    }

    bool removeByName(const std::string& name) { return
        removeFirstIf(
            [&name](const Recipe& r) { return
                r.getName() == name;
            }
        );
    }

void clearAll() {
    // Vacía la lista usando el clear de la lista doblemente ligada DoublyLinkedList<Recipe>::clear();
}

void clear() {
    // Por si en otros lados llaman clear() directamente DoublyLinkedList<Recipe>::clear();
}

int findByName(const std::string& name) const { return
    findIndexIf(
        [&name](const Recipe& r) { return
            r.getName() == name;
        }
    );
}

int findFirstByCategory(Category cat) const { return
    findIndexIf(
        [cat](const Recipe& r) {
            return r.getCategory() == cat;
        }
    );
}

void listAll() const { if (size()
    == 0) {
        std::cout << "No hay recetas registradas.\n"; return;
    }
    for (std::size_t i = 0; i < size(); ++i) { at(i).printDetails();
    }
}

```

```

void listByCategory(Category cat) const { bool
    found = false;
    for (std::size_t i = 0; i < size(); ++i) { if
        (at(i).getCategory() == cat) {
            at(i).printDetails(); found =
                true;
        }
    }
    if (!found) std::cout << "No hay recetas en esa categoría.\n";
}

void sortByNameRecursive() {
    auto comp = [](const Recipe& a, const Recipe& b) { return
        a.getName() < b.getName();
    };
    sortRecursive(comp);
}

void sortByPrepTimeRecursive() {
    auto comp = [](const Recipe& a, const Recipe& b) {
        return a.getPrepTimeMinutes() < b.getPrepTimeMinutes();
    };
    sortRecursive(comp);
}

};
#endif // RECIPE_LIST_HPP

```

doubly_linked_list.hpp

```

#ifndef DOUBLY_LINKED_LIST_HPP
#define DOUBLY_LINKED_LIST_HPP

#include <cstddef> #include
<functional>

// Nodo doblemente ligado genérico template<typename
T>
struct DNode { T
    data; DNode*
    prev; DNode*
    next;

```

```

        explicit DNode(const T& value) : data(value), prev(nullptr), next(nullptr) {}
};

// Lista doblemente ligada genérica template<typename
T>
class DoublyLinkedList {
protected:
    DNode<T>* head;
    DNode<T>* tail;
    std::size_t sz;

    // --- Helpers para merge sort recursivo --- static
    DNode<T>* split(DNode<T>* head) {
        DNode<T>* fast = head;
        DNode<T>* slow = head;
        while (fast && fast->next && fast->next->next) { fast = fast-
            >next->next;
            slow = slow->next;
        }
        DNode<T>* second = slow->next;
        if (second) second->prev = nullptr; slow-
            >next = nullptr;
        return second;
    }

    static DNode<T>* mergeLists(DNode<T>* a, DNode<T>* b,
                                std::function<bool(const T&, const T&)>
comp) {
        if (!a) return b; if (!b)
        return a;

        if (comp(a->data, b->data)) {
            a->next = mergeLists(a->next, b, comp); if (a-
            >next) a->next->prev = a;
            a->prev = nullptr; return a;
        } else {
            b->next = mergeLists(a, b->next, comp); if (b-
            >next) b->next->prev = b;
            b->prev = nullptr; return b;
        }
    }
}

```

```

static DNode<T>* mergeSortRecursive(DNode<T>* node,
                                   std::function<bool(const T&, const
T&> comp) {
    if (!node || !node->next) return node; DNode<T>*
    second = split(node);
    node = mergeSortRecursive(node, comp); second =
    mergeSortRecursive(second, comp); return
    mergeLists(node, second, comp);
}

void rebuildTailFromHead() { tail =
    head;
    while (tail && tail->next) { tail = tail-
        >next;
    }
}

public:
DoublyLinkedList() : head(nullptr), tail(nullptr), sz(0) {}

~DoublyLinkedList(){ clear();
}

bool empty() const { return head == nullptr; } std::size_t
size() const { return sz; }

void clear() {
    DNode<T>* curr = head;
    while (curr) {
        DNode<T>* nxt = curr->next; delete
        curr;
        curr = nxt;
    }
    head = tail = nullptr; sz = 0;
}

void pushBack(const T& value) { DNode<T>*
    node = new DNode<T>(value); if (!head) {
        head = tail = node;
    } else {
        tail->next = node;

```

```

        node->prev = tail; tail =
        node;
    }
    ++sz;
}

T& at(std::size_t index) {
    DNode<T>* curr = head;
    for (std::size_t i = 0; i < index && curr; ++i) { curr = curr->next;
    }
    return curr->data;
}

const T& at(std::size_t index) const { DNode<T>*
    curr = head;
    for (std::size_t i = 0; i < index && curr; ++i) { curr = curr->next;
    }
    return curr->data;
}

int findIndexIf(std::function<bool(const T&)> pred) const { DNode<T>* curr =
    head;
    int idx = 0; while
    (curr) {
        if (pred(curr->data)) return idx; curr =
        curr->next;
        ++idx;
    }
    return -1;
}

bool removeFirstIf(std::function<bool(const T&)> pred) { DNode<T>* curr =
    head;
    while (curr && !pred(curr->data)) { curr = curr-
        >next;
    }
    if (!curr) return false;

    if (curr->prev) curr->prev->next = curr->next; else head =
    curr->next;

    if (curr->next) curr->next->prev = curr->prev; else tail =
    curr->prev;

```

```

        delete curr;
        --SZ;
        return true;
    }

    // Ordenamiento recursivo con comparador explícito
    void sortRecursive(std::function<bool(const T&, const T&)> comp) { if (!head ||
        !head->next) return;
        head = mergeSortRecursive(head, comp);
        rebuildTailFromHead();
    }
};

#endif // DOUBLY_LINKED_LIST_HPP

```

singly_linked_list.hpp

```

#ifndef SINGLY_LINKED_LIST_HPP
#define SINGLY_LINKED_LIST_HPP

#include <cstddef> #include
<functional>

// Nodo simplemente ligado genérico
template<typename T>
struct SNode { T
    data; SNode*
    next;

    explicit SNode(const T& value) : data(value), next(nullptr) {}
};

// Lista simplemente ligada genérica template<typename T>
class SinglyLinkedList { private:
    SNode<T>* head;
    std::size_t sz;

public:
    SinglyLinkedList() : head(nullptr), sz(0) {}

    ~SinglyLinkedList() { clear();

```

```

}

bool empty() const { return head == nullptr; } std::size_t
size() const { return sz; }

void clear() {
    SNode<T>* curr = head;
    while (curr) {
        SNode<T>* nxt = curr->next; delete
        curr;
        curr = nxt;
    }
    head = nullptr; sz =
    0;
}

// Inserta al inicio (no ordenado) void
pushFront(const T& value) {
    SNode<T>* node = new SNode<T>(value);
    node->next = head;
    head = node;
    ++sz;
}

// Inserción ORDENADA usando comparador explícito (comp(a,b) = a<b) bool
insertOrdered(const T& value, std::function<bool(const T&, const
T&)> comp) {
    SNode<T>* node = new SNode<T>(value);

    // Lista vacía o va antes de la cabeza if (!head ||
comp(value, head->data)) {
        node->next = head; head
        = node;
        ++sz; return
        true;
    }

    // Buscar posición donde value va antes de curr->next SNode<T>* curr
    = head;
    while (curr->next && !comp(value, curr->next->data)) { curr = curr-
        >next;
    }

    node->next = curr->next; curr-
    >next = node;
}

```

```

    ++SZ;
    return true;
}

// Elimina el primer elemento que cumpla el predicado bool
removeFirstIf(std::function<bool(const T&> pred) {
    if (!head) return false;

    if (pred(head->data)) {
        SNode<T>* tmp = head;
        head = head->next; delete
        tmp;
        --SZ;
        return true;
    }

    SNode<T>* curr = head;
    while (curr->next && !pred(curr->next->data)) { curr = curr-
        >next;
    }
    if (!curr->next) return false;

    SNode<T>* tmp = curr->next;
    curr->next = tmp->next; delete
    tmp;
    --SZ;
    return true;
}

// Acceso por índice (O(n), pero suficiente para este proyecto) T&
at(std::size_t index) {
    SNode<T>* curr = head;
    for (std::size_t i = 0; i < index && curr; ++i) { curr = curr->next;
    }
    return curr->data; // asume que index es válido
}

const T& at(std::size_t index) const { SNode<T>*
    curr = head;
    for (std::size_t i = 0; i < index && curr; ++i) { curr = curr->next;
    }
    return curr->data;
}

```



```

// Recorrido aplicando función a cada elemento void
forEach(std::function<void(T&)> fn) {
    SNode<T>* curr = head;
    while (curr) {
        fn(curr->data); curr =
            curr->next;
    }
}

void forEach(std::function<void(const T&)> fn) const { SNode<T>*
    curr = head;
    while (curr) { fn(curr->data);
        curr = curr->next;
    }
}
};

#endif // SINGLY_LINKED_LIST_HPP

```

menu.hpp

```

#ifndef MENU_HPP
#define
MENU_HPP

#include "recipe_list.hpp" #include
"storage.hpp"

class Menu {
private:
    RecipeList& recipeList;

    void clearScreen() const; void
    pause() const;

    //Acciones
    void showOptions() const; void
    actionListAll() const;
    void actionListByCategory() const; void
    actionAddRecipe();
    void actionDeleteRecipe(); void
    actionClearAll(); void
    actionSortByName(); void
    actionSortByTime();

```

```

    void actionAddIngredient(); void
    actionRemoveIngredient();
    void actionUpdateIngredientQty(); void
    actionUpdateProcedure(); void
    actionSave();
    void actionLoad();

    //Auxiliares
    Category askCategory() const;

public:
    explicit Menu(RecipeList& list) : recipeList(list) {} void start();
};

#endif // MENU_HPP

```

src/

main.cpp

```

#include "../include/menu.hpp" #include
<iostream>

#ifdef _WIN32
    #include <Windows.h>
#endif

int main() {
    #ifdef _WIN32
        SetConsoleOutputCP(CP_UTF8); // solo en Windows #endif

    RecipeList list; Menu
    menu(list);
    menu.start(); return 0;
}

```

menu.cpp

```

#include "../include/menu.hpp"

```

```

#include <iostream> #include
<limits>

//limpiar pantalla
void Menu::clearScreen() const { #ifdef
_WIN32
    std::system("cls"); #else
    std::system("clear");
#endif
}

//pausar
void Menu::pause() const {
    std::cout << "Presiona ENTER para continuar..."; std::cout.flush();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::cin.get();
}

//menu
void Menu::showOptions() const {
    std::cout << "===== RECETARIO =====\n"
        << " 1) Listar Recetas\n"
        << " 2) Listar por Categoría\n"
        << " 3) Buscar Receta (por nombre)\n"
        << " 4) Agregar Receta\n"
        << " 5) Eliminar Receta (por nombre)\n"
        << " 6) Eliminar TODAS las Recetas\n"
        << " 7) Ordenar por Nombre\n"
        << " 8) Ordenar por Tiempo de Preparación\n"
        << " 9) Agregar Ingrediente (ordenado)\n"
        << "10) Eliminar Ingrediente\n"
        << "11) Modificar Cantidad de Ingrediente\n"
        << "12) Modificar Procedimiento\n"
        << "13) Guardar a Disco\n"
        << "14) Leer de Disco\n"
        << " 0) Salir\n\n"
        << "Opción: ";
}

//Categoría
Category Menu::askCategory() const {
    std::cout << "Categoría [0=Desayuno, 1=Almuerzo, 2=Cena, 3=Navidad]: "; int c; std::cin
    >> c;
}

```

```

        if (std::cin.fail()) {
            std::cin.clear(); std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); c =
            0;
        }
        if (c < 0 || c > 3) c = 0; return
        static_cast<Category>(c);
    }

    //Listar recetas
    void Menu::actionListAll() const { clearScreen();
        if (recipeList.size() == 0) {
            std::cout << "No hay recetas registradas.\n"; return;
        }
        for (std::size_t i = 0; i < recipeList.size(); ++i) { recipeList.at(i).printDetails(); //imprimir recetas con
            ingredientes
        }
    }

    //Listar por categorias
    void Menu::actionListByCategory() const { clearScreen();
        Category c = askCategory(); bool
        found = false;
        for (std::size_t i = 0; i < recipeList.size(); ++i) { if
            (recipeList.at(i).getCategory() == c) {
                recipeList.at(i).printDetails(); found = true;
            }
        }
        if (!found){
            std::cout << "No hay recetas en esta categoría.\n";
        }
    }

    //Agregar receta
    void Menu::actionAddRecipe(){
        clearScreen();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::string name, author,
        procedure;
    }

```

```

int prep;

std::cout << "Nombre de la receta: ";
std::getline(std::cin, name); std::cout <<
"Nombre del autor: "; std::getline(std::cin,
author); Category cat = askCategory();
std::cout << "Tiempo de preparación (min): "; std::cin >>
prep;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::cout <<
"Procedimiento (una línea): ";
std::getline(std::cin, procedure);

Recipe r(name, author, cat, prep, procedure); std::string
resp;
std::cout << "¿Desea agregar ingredientes ahora? (s/n): "; std::getline(std::cin, resp);

if (!resp.empty() && (resp[0] == 's' || resp[0] == 'S')) { int n;
    std::cout << "¿Cuántos ingredientes desea agregar?: "; std::cin >> n;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); for (int i = 0; i < n;
    ++i) {
        std::string ingName, unit; double qty;
        std::cout << "Ingrediente #" << (i + 1) << " - Nombre: "; std::getline(std::cin, ingName);
        std::cout << "Cantidad: "; std::cin >> qty;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        std::cout << "Unidad: "; std::getline(std::cin, unit);
        r.addIngredientOrdered(Ingredient(ingName, qty, unit));
    }
}

if (recipeList.add(r)) std::cout << "Receta agregada.\n"; else std::cout <<
"La lista de recetas está llena.\n";
}

//Eliminar receta
void Menu::actionDeleteRecipe() { clearScreen();

```

```

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::string name;
std::cout << "Nombre de la receta a eliminar: ";
std::getline(std::cin, name);
if (recipeList.removeByName(name)) std::cout << "Receta eliminada.\n"; else std::cout <<
"No se encontró la receta.\n";
}

//Vaciar recetario
void Menu::actionClearAll() { char
    confirma;

    std::cout << "¿Estás seguro de eliminar TODAS las recetas? (s/n): "; if (!(std::cin >>
    confirma)) {
        std::cin.clear(); std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Entrada inválida. Operación cancelada.\n";
        return;
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    if (confirma == 's' || confirma == 'S') {
        recipeList.clear(); // O clearAll(), según como lo tengas std::cout
        << "→ Todas las recetas han sido eliminadas.\n";
    } else {
        std::cout << "Operación cancelada.\n";
    }
}

//Ordenar por nombre
void Menu::actionSortByName() { clearScreen();
    recipeList.sortByNameRecursive();
    std::cout << "Recetas ordenadas por nombre.\n";
}

//Ordenar por tiempo de preparacion void
Menu::actionSortByTime() {
    clearScreen(); recipeList.sortByPrepTimeRecursive();
    std::cout << "Recetas ordenadas por tiempo de preparación.\n";
}

void Menu::actionAddIngredient() {

```

```

// Limpia el '\n' que quedó después de leer la opción del menú
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

std::string recipeName;
std::cout << "Nombre de la receta a la que deseas agregar un ingrediente: ";
";
std::getline(std::cin, recipeName);

int idx = recipeList.findByName(recipeName); if (idx == -
1) {
    std::cout << "× No se encontró la receta.\n"; return;
}

Recipe& r = recipeList.at(idx);

std::string ingName, unit; double
qty;

std::cout << "Nombre del ingrediente: "; std::getline(std::cin,
ingName);

std::cout << "Cantidad (numérica): "; while
(!std::cin >> qty) {
    std::cin.clear(); std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::cout << "Valor inválido. Ingresar un número: ";
}
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

std::cout << "Unidad: "; std::getline(std::cin, unit);

if (r.addIngredientOrdered(Ingredient(ingName, qty, unit))) { std::cout << "→
    Ingrediente agregado en orden.\n";
} else {
    std::cout << "× No se pudo agregar el ingrediente.\n";
}
}

//Eliminar ingrediente
void Menu::actionRemoveIngredient() { clearScreen();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

```

```

std::string recipeName, ingName; std::cout <<
"Nombre de la receta: "; std::getline(std::cin,
recipeName);
int idx = recipeList.findByName(recipeName);
if (idx == -1) { std::cout << "Receta no encontrada.\n"; return; }

std::cout << "Ingrediente a eliminar: "; std::getline(std::cin,
ingName);
if (recipeList.at(idx).removeIngredientByName(ingName)) std::cout <<
    "Ingrediente eliminado.\n";
else
    std::cout << "Ingrediente no encontrado.\n";
}

//Modificar cantidad de ingrediente void
Menu::actionUpdateIngredientQty(){
    clearScreen(); std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::string
    recipeName, ingName;
    double qty;
    std::cout << "Nombre de la receta: "; std::getline(std::cin,
    recipeName);
    int idx = recipeList.findByName(recipeName);
    if (idx == -1) { std::cout << "Receta no encontrada.\n"; return; }

    std::cout << "Nombre del ingrediente: ";
    std::getline(std::cin, ingName); std::cout <<
    "Nueva cantidad: "; std::cin >> qty;
    if (recipeList.at(idx).updateIngredientQuantity(ingName, qty)) std::cout << "Cantidad
        actualizada.\n";
    else
        std::cout << "Ingrediente no encontrado.\n";
}

//Modificar proceso
void Menu::actionUpdateProcedure() { clearScreen();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::string recipeName,
    proc;
    std::cout << "Nombre de la receta: "; std::getline(std::cin,
    recipeName);
    int idx = recipeList.findByName(recipeName);
    if (idx == -1) { std::cout << "Receta no encontrada.\n"; return; } std::cout << "Nuevo
    procedimiento (una línea): ";

```



```

std::getline(std::cin, proc);
recipeList.at(idx).setProcedure(proc); std::cout <<
"Procedimiento actualizado.\n";
}

//Guardar archivo
void Menu::actionSave() {
    clearScreen();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); std::string path;
    std::cout << "Ruta de archivo para guardar: ";
    std::getline(std::cin, path);
    if (Storage::saveToFile(recipeList, path)) std::cout << "Guardado.\n"; else std::cout <<
    "Error al guardar el archivo.\n";
}

//Cargar archivo
void Menu::actionLoad() {
    clearScreen();
    // Limpia el '\n' que quedó después de leer la opción del menú
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    std::string path;
    std::cout << "Nombre del archivo a leer: ";
    std::getline(std::cin, path);

    if (path.empty()) {
        std::cout << "× Nombre de archivo vacío.\n"; return;
    }

    try {
        if (Storage::loadFromFile(recipeList, path)) {
            std::cout << "→ Recetario cargado correctamente desde " << path
<< ".\n";
        } else {
            std::cout << "× No se pudo cargar el recetario (archivo inexistente o formato
inválido).\n";
        }
    } catch (const std::exception& e) {
        std::cout << "× Error al leer desde el archivo: " << e.what() <<
"\n";
    }
}

```

```

//Iniciar menu
void Menu::start() { int op
    = -1; while (true) {
        clearScreen();           // limpia antes de mostrar el menú
        showOptions();
        std::cin >> op;
        if (std::cin.fail()) {
            std::cin.clear(); std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');

            std::cout << "Entrada inválida.\n"; pause();
            continue;

        }

        switch (op) {
            case 1: actionListAll(); break;
            case 2: actionListByCategory(); break; case 3: {
                //Buscar receta clearScreen();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(),

                std::string name;
                std::cout << "Nombre de la receta a buscar: ";
                std::getline(std::cin, name);
                int idx = recipeList.findByName(name);
                if (idx != -1) std::cout << "Encontrada: " <<
recipeList.at(idx).toString() << "\n";
                else std::cout << "No se encontró.\n"; break;
            }

            case 4: actionAddRecipe(); break; case 5:
            actionDeleteRecipe(); break; case 6:
            actionClearAll(); break; case 7:
            actionSortByName(); break; case 8:
            actionSortByTime(); break; case 9:
            actionAddIngredient(); break;
            case 10: actionRemoveIngredient(); break; case 11:
            actionUpdateIngredientQty(); break; case 12:
            actionUpdateProcedure(); break; case 13:
            actionSave(); break;
            case 14: actionLoad(); break;
            case 0: std::cout << "¡Hasta luego!\n"; return;
        }
    }
}

```

```

        default: std::cout << "Opción inválida.\n"; break;
    }
    std::cout << "\n\n";
    pause(); // espera antes de refrescar el menú
}
}

```

recipe.cpp

```
#include "../include/recipe.hpp"
```

```
#include <functional> #include
<iostream>
```

```

bool Recipe::addIngredientOrdered(const Ingredient& ing) {
    // Comparador explícito: ordena por nombre de ingrediente auto comp =
    [](const Ingredient& a, const Ingredient& b) {
        return a.getName() < b.getName();
    };
    return ingredients.insertOrdered(ing, comp);
}

```

```

bool Recipe::removeIngredientByName(const std::string& ingName) { return
    ingredients.removeFirstIf(
        [&ingName](const Ingredient& ing) { return
            ing.getName() == ingName;
        }
    );
}

```

```

void Recipe::clearIngredients() { ingredients.clear();
}

```

```

bool Recipe::updateIngredientQuantity(const std::string& ingName, double newQty) {
    bool updated = false;
    ingredients.forEach(
        [&updated, &ingName, newQty](Ingredient& ing) { if
            (ing.getName() == ingName) {
                ing.setQuantity(newQty);
            }
        }
    );
    return updated;
}

```

```

        updated = true;
    }
}
);
return updated;
}

void Recipe::printDetails() const {
    std::cout << "\n===== \n";
    std::cout << "Nombre: " << name << "\n"; std::cout <<
    "Autor: " << authorName << "\n";
    std::cout << "Categoría: " << categoryToString(category) << "\n"; std::cout << "Tiempo de
    preparación: " << prepTimeMinutes << " min\n"; std::cout << "Procedimiento: " <<
    procedure << "\n";

    if (ingredients.size() == 0) {
        std::cout << "Ingredientes: ninguno.\n";
    } else {
        std::cout << "Ingredientes (" << ingredients.size() << "):\n"; ingredients.forEach(
            [](const Ingredient& ing) {
                std::cout << " - " << ing.toString() << "\n";
            }
        );
    }
}
}

```

storage.cpp

```

#include "../include/storage.hpp" #include
<fstream>
#include <sstream> #include
<limits>

bool Storage::saveToFile(const RecipeList& list, const std::string& path) { std::ofstream out(path);
    if (!out) {
        std::cout << "× No se pudo abrir el archivo para escritura.\n"; return false;
    }

    out << list.size() << "\n";

    for (std::size_t i = 0; i < list.size(); ++i) { const Recipe& r =
        list.at(i);
    }
}

```

```

    out << r.getName() << "\n";
    out << r.getAuthorName() << "\n";
    out << categoryToString(r.getCategory()) << "\n"; out <<
    r.getPrepTimeMinutes() << "\n";
    out << r.getProcedure() << "\n";

    int ingCount = r.getIngredientCount(); out <<
    ingCount << "\n";
    for (int j = 0; j < ingCount; ++j) {
        const Ingredient& ing = r.getIngredientAt(j); out <<
        ing.getName() << "|" << ing.getQuantity()
        << "|" << ing.getUnit() << "\n";
    }
}

return true;
}

static bool splitLine(const std::string& line, char sep, std::string& a, std::string& b, std::string& c)
{
    size_t p1 = line.find(sep);
    if (p1 == std::string::npos) return false; size_t p2 =
    line.find(sep, p1 + 1);
    if (p2 == std::string::npos) return false;
    a = line.substr(0, p1);
    b = line.substr(p1 + 1, p2 - p1 - 1);
    c = line.substr(p2 + 1); return
    true;
}

bool Storage::loadFromFile(RecipeList& list, const std::string& path) { std::ifstream in(path);
    if (!in) {
        std::cout << "x No se pudo abrir el archivo para lectura.\n"; return false;
    }

    list.clear(); // o clearAll() int n;
    in >> n; in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    for (int i = 0; i < n; ++i) {
        std::string name, author, catStr, procedure;

```

```

int prep;

std::getline(in, name); std::getline(in, author);
std::getline(in, catStr);

in >> prep; in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

std::getline(in, procedure);

Category cat = categoryFromString(catStr); Recipe r(name,
author, cat, prep, procedure);

int m; in
>> m;
in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

for (int j = 0; j < m; ++j) { std::string
    line; std::getline(in, line); if
    (line.empty()) {
        -j;
        continue;
    }

    auto p1 = line.find('|');
    auto p2 = line.find('|', p1 + 1); std::string ingName =
    line.substr(0, p1);
    double qty = std::stod(line.substr(p1 + 1, p2 - p1 - 1)); std::string unit =
    line.substr(p2 + 1);

    r.addIngredientOrdered(Ingredient(ingName, qty, unit));
}

list.add(r);
}

return true;
}

```

recetas.txt

10

Hotcakes de Avena

Chef

Papu

Desayuno

15

Mezcla todo y cocina en sartén.

10

Avena|1|taza

Leche|1|taza

Huevo|1|pieza

Banana|1|pieza

Canela|1|cucharadita

Polvo para hornear|1|cucharadita

Sal|1|pizca

Aceite vegetal|1|cucharada

Miel|2|cucharadas

Extracto de vainilla|1|cucharadita

Ensalada César

Chef

Papu

Almuerzo

20

Combina ingredientes y adereza al gusto.

10

Lechuga romana|1|unidad

Pechuga de pollo|200|gramos

Pan tostado|50|gramos

Queso parmesano|30|gramos

Aceite de oliva|2|cucharadas

Jugo de limón|1|cucharada

Mostaza|1|cucharadita

Sal|1|pizca

Pimienta|1|pizca

Huevo|1|pieza

Sopa de Tomate

Chef Papu

Almuerzo

25

Hierve los ingredientes y licúa hasta obtener textura suave. 10

Tomate|5|piezas

Cebolla|0.5|pieza

Ajo|2|dientes

Aceite de oliva|2|cucharadas

Caldo de pollo|2|tazas

Sal|1|cucharadita

Pimienta|0.5|cucharadita

Azúcar|0.5|cucharadita

Crema|0.25|taza

Perejil|1|cucharada

Tacos de Asada

Chef Papu

Almuerzo

30

Cocina la carne y sirve con tortillas y salsas.

10

Carne de res|500|gramos

Tortillas|10|piezas

Cilantro|0.5|taza

Cebolla|0.5|taza

Sal|1|cucharadita

Pimienta|0.5|cucharadita

Aceite vegetal|2|cucharadas

Limón|2|piezas

Salsa roja|3|cucharadas

Aguacate|1|pieza

Spaghetti Boloñesa

Chef Papu

Cena

40

Cocina la pasta y mezcla con la salsa de carne.

10

Spaghetti|250|gramos

Carne molida|300|gramos

Tomate|3|piezas

Cebolla|1|pieza

Ajo|2|dientes

Aceite de oliva|2|cucharadas

Sal|1|cucharadita

Pimienta|0.5|cucharadita

Orégano|1|cucharadita

Queso parmesano|30|gramos

Pizza Margarita

Chef Papu

Cena

35

Hornea la pizza hasta que el queso se derrita.

10

Masa para pizza|1|unidad

Salsa de tomate|0.5|taza

Queso mozzarella|200|gramos

Aceite de oliva|1|cucharada

Albahaca|5|hojas

Tomate|1|pieza

Sal|1|pizca

Pimienta|1|pizca

Orégano|0.5|cucharadita

Queso parmesano|20|gramos

Galletas de Chocolate

Chef

Papu

Navidad

25

Mezcla los ingredientes, forma bolitas y hornea.

10

Harina|2|tazas

Mantequilla|100|gramos

Azúcar|0.75|taza

Huevo|2|piezas

Cacao en polvo|0.5|taza

Polvo para hornear|1|cucharadita

Sal|0.25|cucharadita

Chispas de chocolate|0.5|taza

Leche|2|cucharadas

Vainilla|1|cucharadita

Pavo al

Horno Chef

Papu

Navidad

180

Hornea el pavo cubierto con mantequilla y especias.

10

Pavo entero|1|unidad

Mantequilla|200|gramos

Ajo|5|dientes

Sal|2|cucharaditas

Pimienta|1|cucharadita

Hierbas finas|2|cucharadas

Cebolla|1|pieza

Zanahoria|2|piezas

Apio|2|tallos

Vino blanco|1|taza

Ceviche de

Pescado Chef

Papu Almuerzo

25

Mezcla los ingredientes y deja marinar 20 minutos.

10

Pescado blanco|300|gramos

Jugo de limón|1|taza

Cebolla morada|0.5|taza

Tomate|1|pieza

Pepino|0.5|taza

Cilantro|0.25|taza

Sal|1|cucharadita

Pimienta|0.5|cucharadita

Chile serrano|1|pieza

Aguacate|1|pieza

Pastel de Zanahoria

Chef Papu

Navida

d 60

Hornea hasta que al insertar un palillo, salga limpio. 10

Zanahoria rallada|2|tazas

Harina|2|tazas

Huevo|3|piezas

Azúcar|1|taza

Aceite vegetal|1|taza

Polvo para hornear|2|cucharaditas

Canela|1|cucharadita

Nuez picada|0.5|taza

Sal|0.25|cucharadita

Vainilla|1|cucharadita

PRUEBAS

Para la mayoría de las pruebas se utilizará un archivo previamente echo: "recetas.txt" para comprobar el comportamiento y funcionamiento del programa donde vienen 10 recetas con 10 ingredientes cada una. También para comprobar el resultado (positivo o negativo) de cada funcionalidad se usará la opción 1) Listar Recetas.

1. Listar Recetas

La función **1) Listar Recetas**, tiene como objetivo mostrar en pantalla todas las recetas almacenadas.

Se pone a prueba que el sistema:

- Recorra la lista doblemente ligada completa de recetas.
- Muestre para cada receta:
 - Nombre
 - Autor
 - Categoría
 - Tiempo de preparación en minutos
 - Procedimiento
 - Lista de ingredientes (nombre, cantidad y unidad), respetando el orden alfabético.
- Indique apropiadamente cuando no hay recetas registradas.

```
=====
Nombre: Pastel de Zanahoria
Autor: Chef Papu
Categoría: Desayuno
Tiempo de preparación60min
Procedimiento: Hornea hasta que al insertar un palillo, salga
limpio.
Ingredientes (10):
- Aceite vegetal : 1 taza
- Azúcar : 1 taza
- Canela : 1 cucharadita
- Harina : 2 tazas
- Huevo : 3 piezas
- Nuez picada : 0.5 taza
- Polvo para hornear : 2 cucharaditas
- Sal : 0.25 cucharadita
- Vainilla : 1 cucharadita
- Zanahoria rallada : 2 tazas
```

2. Listar por categoría

La función **2) Listar por Categoría**, tiene como objetivo filtrar y mostrar únicamente las recetas que pertenecen a una categoría específica.

Se pone a prueba que el sistema:

- Solicite al usuario una categoría válida (0-3).
- Recorra la lista de recetas y solo muestre aquellas cuyo campo Category coincide.
- Incluya en el listado:
 - Datos generales de la receta
 - Ingredientes correspondientes
- Muestre un mensaje claro cuando no exista ninguna receta en la categoría seleccionada.

I. Categoría 0: Desayunos

```
Nombre: Hotcakes de Avena
Autor: Chef Papu
Categoría: Desayuno
Tiempo de preparación15min
'Procedimiento: Mezcla todo y cocina en sartén.
Ingredientes (10):
- Aceite vegetal : 1 cucharada
- Avena : 1 taza
- Banana : 1 pieza
- Canela : 1 cucharadita
- Extracto de vainilla : 1 cucharadita
- Huevo : 1 pieza
- Leche : 1 taza
- Miel : 2 cucharadas
- Polvo para hornear : 1 cucharadita
- Sal : 1 pizca
```

```
=====
```

II. Categoría 1: Almuerzos

```

Nombre: Tacos de Asada
Autor: Chef Papu
Categoría: Almuerzo
Tiempo de preparación30min
'Procedimiento: Cocina la carne y sirve con tortillas y salsas.
Ingredientes (10):
- Aceite vegetal : 2 cucharadas
- Aguacate : 1 pieza
- Carne de res : 500 gramos
- Cebolla : 0.5 taza
- Cilantro : 0.5 taza
- Limón : 2 piezas
- Pimienta : 0.5 cucharadita
- Sal : 1 cucharadita
- Salsa roja : 3 cucharadas
- Tortillas : 10 piezas
=====

```

III. Categoría 2: Cenas

```

Nombre: Spaghetti Bolonesa
Autor: Chef Papu
Categoría: Cena
Tiempo de preparación40min
'Procedimiento: Cocina la pasta y mezcla con la salsa de carne.
Ingredientes (10):
- Aceite de oliva : 2 cucharadas
- Ajo : 2 dientes
- Carne molida : 300 gramos
- Cebolla : 1 pieza
- Orégano : 1 cucharadita
- Pimienta : 0.5 cucharadita
- Queso parmesano : 30 gramos
- Sal : 1 cucharadita
- Spaghetti : 250 gramos
- Tomate : 3 piezas

```

IV. Categoría 3: Navideño

```

Nombre: Galletas
Autor: Alan
Categoría: Navideño
Tiempo de preparación25min
'Procedimiento: mezcla los ingredientes, forma bolitas y hornea
Ingredientes (10):
- azucar : 0.75 taza
- cacao en polvo : 0.5 taza
- chipas de chocolate : 0.5 taza
- harina : 2 tazas
- huevo : 2 piezas
- leche : 2 cucharadas
- mantequilla : 100 gramos
- polvo para hornear : 1 taza
- sal : 0.25 cucharadita
- vainilla : 1 cucharadita

```


3. Buscar receta

La función asociada a **3) Buscar Receta (por nombre)** tiene como objetivo localizar una receta específica dentro de la lista.

Se pone a prueba que el sistema:

- Solicite el **nombre exacto** del platillo.
- Recorra la lista doblemente ligada utilizando la operación `findByName`.
- En caso de encontrar la receta:
 - Muestre todos sus datos e ingredientes.
- En caso de no encontrarla.
 - Informe correctamente que la receta no existe.

```
Nombre de la receta a buscar: Sopa de Tomate
Encontrada: Receta{nombre='Sopa de Tomate', autor='Chef Papu', ca
tegoria='Almuerzo', preparacion=25 min}
```

```
Presiona ENTER para continuar...|
```

```
Nombre de la receta a buscar: papu
No se encontró.
```

```
Presiona ENTER para continuar...|
```

4. Eliminar receta

La función **5) Eliminar Receta** tiene como objetivo borrar una receta específica del recetario.

Se pone a prueba que el sistema:

- Solicite el nombre de la receta a eliminar.
- Use `removeByName` sobre la lista doblemente ligada.
- Elimine tanto la receta como su lista de ingredientes asociada
- Muestre:
 - Un mensaje de éxito cuando la receta sí existía y fue eliminada.
 - Un mensaje de error cuando el nombre no corresponde a ninguna receta.

```
Nombre de la receta a eliminar: enchiladas verdes  
Receta eliminada.
```

```
Presiona ENTER para continuar...
```

```
Nombre de la receta a eliminar: papu  
No se encontró la receta.
```

```
Presiona ENTER para continuar...|
```

5. Eliminar todas las recetas

La función **5) Eliminar Todas las Recetas** tiene como objetivo vaciar completamente el recetario.

Se pone a prueba que el sistema:

- Llame al método `clear()/clearAll()` de `RecipeList`.
- Libere todos los nodos de la lista doblemente ligada y todo su contenido.
- Posteriormente, al intentar listar recetas, el programa indique que **no hay registros**.

```
¿Estás seguro de eliminar TODAS las recetas? (s/n): s|
```

```
→ Todas las recetas han sido eliminadas.
```

```
Presiona ENTER para continuar...|
```

```
No hay recetas registradas.
```

```
Presiona ENTER para continuar...|
```

6. Agregar receta

La función **4) Agregar Receta** tiene como objetivo registrar una nueva receta con todos sus datos en el recetario.

Se pone a prueba que el sistema:

- Solicite al usuario el nombre del platillo, el nombre del autor, la categoría, el tiempo de preparación y el procedimiento.
- Valide que la categoría seleccionada sea una de las permitidas (0–3) mediante la función `askCategory`.
- Permita al usuario decidir si desea capturar ingredientes en ese momento. En caso afirmativo, para cada ingrediente solicite nombre, cantidad numérica y unidad de medida.

- Inserte cada ingrediente utilizando la operación `addIngredientOrdered`, manteniendo la lista simplemente ligada de ingredientes ordenada alfabéticamente por nombre.
- Agregue la nueva receta a la lista doblemente ligada de recetas mediante `RecipeList::add (pushBack)`.
- Permita verificar la inserción utilizando 1) Listar Recetas y 2) Listar por Categoría, mostrando la nueva receta con todos sus datos.

```
Nombre de la receta: enchiladas verdes
Nombre del autor: yadira
Categoría [0=Desayuno, 1=Almuerzo, 2=Cena, 3=Navidad]: 1
Tiempo de preparación (min): 30
Procedimiento (una línea): cocer pechugas de pollo en agua, con e
l caldo resultante licuar tomatillos con sal para la salsa y arma
r las enchiladas
¿Desea agregar ingredientes ahora? (s/n): s
¿Cuántos ingredientes desea agregar?: 10
Ingrediente #1 - Nombre: sal
Cantidad: 0.25
```

7. Ordenar por nombre

La función **7) Ordenar por Nombre** tiene como objetivo reordenar la lista de recetas en orden alfabético ascendente según el nombre del platillo.

Se pone a prueba que el sistema:

- Llame al método `sortByNameRecursive` sobre la lista doblemente ligada de recetas.
- No pierda ni duplique recetas durante el proceso de ordenamiento.
- Mantenga intactos los datos internos de cada receta (autor, categoría, tiempo de preparación, procedimiento e ingredientes).
- Al utilizar posteriormente 1) Listar Recetas, muestre las recetas ordenadas de la A a la Z por el campo nombre.

```
Nombre: Ceviche de Pescado
Autor: Chef Papu
Categoría: Almuerzo
Tiempo de preparación: 25min
Procedimiento: Mezcla los ingredientes y deja marinar 20 minutos
Ingredientes (10):
- Aguacate : 1 pieza
- Cebolla morada : 0.5 taza
- Chile serrano : 1 pieza
- Cilantro : 0.25 taza
- Jugo de limón : 1 taza
- Pepino : 0.5 taza
- Pescado blanco : 300 gramos
- Pimienta : 0.5 cucharadita
- Sal : 1 cucharadita
- Tomate : 1 pieza
```

8. Ordenar por tiempo de preparación

La función **8) Ordenar por Tiempo de Preparación** tiene como objetivo reordenar la lista de recetas de menor a mayor tiempo de preparación (en minutos).

Se pone a prueba que el sistema:

- Llame al método `sortByPrepTimeRecursive` sobre la lista doblemente ligada de recetas.
- Ordene las recetas de forma ascendente según su campo `prepTimeMinutes`.
- No elimine ni duplique recetas al aplicar el algoritmo recursivo de ordenamiento.
- Al utilizar 1) Listar Recetas después del ordenamiento, muestre las recetas ordenadas desde la de menor tiempo de preparación hasta la de mayor.

```
Nombre: Hotcakes de Avena
Autor: Chef Papu
Categoría: Desayuno
Tiempo de preparación15min
'Procedimiento: Mezcla todo y cocina en sartén.
Ingredientes (10):
- Aceite vegetal : 1 cucharada
- Avena : 1 taza
- Banana : 1 pieza
- Canela : 1 cucharadita
- Extracto de vainilla : 1 cucharadita
- Huevo : 1 pieza
- Leche : 1 taza
- Miel : 2 cucharadas
- Polvo para hornear : 1 cucharadita
- Sal : 1 pizca
```

9. Agregar ingrediente (ordenado)

La función **G) Agregar Ingrediente (ordenado)** tiene como objetivo añadir un nuevo ingrediente a una receta existente, manteniendo el orden alfabético por nombre dentro de la lista simplemente ligada de ingredientes.

Se pone a prueba que el sistema:

- Solicite el nombre de la receta a modificar.
- Utilice `recipeList.findByName` para localizar la receta dentro de la lista doblemente ligada.
- En caso de no encontrar la receta, muestre un mensaje claro indicando que la receta no existe y termine la operación.
- En caso de encontrarla, solicite nombre, cantidad numérica y unidad del ingrediente a agregar.

- Inserte el nuevo ingrediente utilizando `Recipe::addIngredientOrdered`, manteniendo la lista de ingredientes ordenada alfabéticamente por nombre.
- Permita verificar el resultado utilizando 1) Listar Recetas, observando que el ingrediente aparece en la posición correcta dentro de la lista de ingredientes de la receta.

```
Nombre de la receta: Sopa de Tomate
Nombre del ingrediente: agua
Cantidad: 1
Unidad: taza
Ingrediente agregado (en orden alfabético).
```

```
Presiona ENTER para continuar...|
```

```
Nombre: Sopa de Tomate
Autor: Chef Papu
Categoría: Almuerzo
Tiempo de preparación25min
'Procedimiento: Hierve los ingredientes y licúa hasta obtener tex
tura suave.
Ingredientes (11):
- Aceite de oliva : 2 cucharadas
- Ajo : 2 dientes
- Azúcar : 0.5 cucharadita
- Caldo de pollo : 2 tazas
- Cebolla : 0.5 pieza
- Crema : 0.25 taza
- Perejil : 1 cucharada
- Pimienta : 0.5 cucharadita
- Sal : 1 cucharadita
- Tomate : 5 piezas
- agua : 1 taza
```

10. Eliminar ingrediente

La función **10) Eliminar Ingrediente** tiene como objetivo borrar un ingrediente específico de la lista de ingredientes de una receta.

Se pone a prueba que el sistema:

- Solicite el nombre de la receta donde se realizará la eliminación.
- Busque la receta utilizando `recipeList.findByName`.
- Si la receta no existe, muestre el mensaje "Receta no encontrada." y termine la operación sin cambios.
- Si la receta existe, solicite el nombre del ingrediente a eliminar.
- Llame a `Recipe::removeIngredientByName` y, en caso de éxito, muestre un mensaje de confirmación indicando que el ingrediente fue eliminado.
- En caso de que el ingrediente no exista, muestre el mensaje "Ingrediente no encontrado."
- Permita verificar que el ingrediente ya no aparece al listar la receta con 1) Listar Recetas.

```
Nombre de la receta: Sopa de Tomate
Ingrediente a eliminar: agua
Ingrediente eliminado.
```

```
Presiona ENTER para continuar...|
```

```
Ingredientes (10):
- Aceite de oliva : 2 cucharadas
- Ajo : 2 dientes
- Azúcar : 0.5 cucharadita
- Caldo de pollo : 2 tazas
- Cebolla : 0.5 pieza
- Crema : 0.25 taza
- Perejil : 1 cucharada
- Pimienta : 0.5 cucharadita
- Sal : 1 cucharadita
- Tomate : 5 piezas
```

11. Modificar cantidad de ingrediente

La función **11) Modificar Cantidad de Ingrediente** tiene como objetivo actualizar la cantidad numérica asociada a un ingrediente específico dentro de una receta.

Se pone a prueba que el sistema:

- Solicite el nombre de la receta que se desea modificar.
- Localice la receta en la lista usando `recipeList.findByName`.
- Si la receta no existe, muestre el mensaje "Receta no encontrada." y no intente modificar ingredientes.
- Si la receta existe, solicite el nombre del ingrediente cuya cantidad se quiere modificar y la nueva cantidad numérica.
- Utilice `Recipe::updateIngredientQuantity` para aplicar el cambio sobre la lista de ingredientes.
- En caso de éxito, muestre el mensaje "Cantidad actualizada."
- Si el ingrediente no se encuentra, muestre el mensaje "Ingrediente no encontrado."
- Permita comprobar la modificación utilizando 1) Listar Recetas, verificando que la cantidad mostrada para ese ingrediente coincide con el nuevo valor.

```
Ingredientes (10):  
- Aceite de oliva : 2 cucharadas  
- Ajo : 2 dientes  
- Azúcar : 0.5 cucharadita  
- Caldo de pollo : 2 tazas  
- Cebolla : 0.5 pieza  
- Crema : 0.25 taza  
- Perejil : 1 cucharada  
- Pimienta : 0.5 cucharadita  
- Sal : 1 cucharadita  
- Tomate : 7 piezas
```

12. Modificar procedimiento

La función **12) Modificar Procedimiento** tiene como objetivo actualizar el texto del procedimiento de una receta ya registrada.

Se pone a prueba que el sistema:

- Solicite el nombre de la receta cuyo procedimiento se desea modificar.
- Busque la receta mediante `recipeList.findByName`.
- Si la receta no existe, muestre el mensaje "Receta no encontrada." y no realice cambios.

- Si la receta existe, solicite el nuevo procedimiento (en una línea).
- Actualice el campo correspondiente llamando a `Recipe::setProcedure`.
- Muestre un mensaje de confirmación indicando que el procedimiento fue actualizado.
- Permita comprobar el cambio utilizando 1) Listar Recetas, verificando que el procedimiento mostrado corresponde al nuevo texto capturado.

```
Nombre de la receta: Galletas
Nuevo procedimiento (una línea): derretir la mantequilla y mezclarla con los demas ingredientes para formar bolitas y hornearlas
Procedimiento actualizado.
```

```
Presiona ENTER para continuar...|
```

```
Nombre: Galletas
Autor: Alan
Categoría: Navideño
Tiempo de preparación25min
'Procedimiento: derretir la mantequilla y mezclarla con los demas
ingredientes para formar bolitas y hornearlas
Ingredientes (10):
```

13. Guardar a disco

La función **13) Guardar a Disco** tiene como objetivo persistir en un archivo de texto todo el contenido del recetario (recetas e ingredientes) para poder recuperarlo posteriormente.

Se pone a prueba que el sistema:

- Limpie la pantalla y solicite al usuario una ruta o nombre de archivo para guardar.
- Llame a `Storage::saveToFile` pasando la lista actual de recetas y la ruta indicada.
- En caso de que el archivo se abra y escriba correctamente, muestre el mensaje "Guardado." (y, en su caso, el mensaje interno de éxito de la función).
- En caso de error al abrir o escribir el archivo, muestre un mensaje de error claro ("Error al guardar el archivo." o "× No se pudo abrir el archivo para escritura.").
- Permita verificar el funcionamiento guardando el recetario y cargándolo después con la opción 14) Leer de Disco, comprobando que los datos coinciden al listar recetas.

```
Ruta de archivo para guardar: recetas2.txt
Guardado.
```

```
Presiona ENTER para continuar...
```

```
≡ recetas.txt
```

```
≡ recetas2.txt
```

14. Leer de disco

La función **14) Leer de Disco** tiene como objetivo reconstruir en memoria el recetario a partir de un archivo previamente guardado en el formato del sistema.

Se pone a prueba que el sistema:

- Solicite al usuario el nombre o ruta del archivo a leer.
- Verifique que el nombre de archivo no esté vacío; en caso contrario, muestre un mensaje de error y no continúe con la lectura.
- Llame a `Storage::loadFromFile`, que debe:
- Vaciar primero la lista actual de recetas llamando a `RecipeList::clear()`.
- Leer el número total de recetas, los datos generales de cada una (nombre, autor, categoría, tiempo, procedimiento).
- Leer para cada receta el número de ingredientes y reconstruir su lista de ingredientes utilizando `addIngredientOrdered`.
- En caso de lectura exitosa, muestre un mensaje de confirmación indicando que el recetario fue cargado correctamente.
- En caso de que el archivo no exista o el formato sea inválido, muestre mensajes de error adecuados sin cerrar inesperadamente el programa.
- Permita comprobar el resultado utilizando 1) Listar Recetas y 2) Listar por Categoría, verificando que las recetas e ingredientes coinciden con las que se habían guardado previamente.

```
Nombre del archivo a leer: recetas.txt
→ Recetario cargado correctamente desde 'recetas.txt'.
```

```
Presiona ENTER para continuar...|
```

```
Nombre del archivo a leer: receta
x No se pudo abrir el archivo 'receta' para lectura.
x No se pudo cargar el recetario (archivo inexistente o formato inválido).
```

```
Presiona ENTER para continuar...|
```

```
=====
Nombre: Pastel de Zanahoria
Autor: Chef Papu
Categoría: Desayuno
Tiempo de preparación: 60 min
Procedimiento: Hornea hasta que al insertar un palillo, salga limpio.
Ingredientes (10):
  - Aceite vegetal : 1 taza
  - Azúcar : 1 taza
  - Canela : 1 cucharadita
  - Harina : 2 tazas
  - Huevo : 3 piezas
  - Nuez picada : 0.5 taza
  - Polvo para hornear : 2 cucharaditas
  - Sal : 0.25 cucharadita
  - Vainilla : 1 cucharadita
  - Zanahoria rallada : 2 tazas
```

```
Presiona ENTER para continuar...|
```

PUESTA EN MARCHA

Manual de Instalación

El presente apartado describe los requisitos y pasos necesarios para instalar y compilar el sistema “Recetario Digital” en un entorno de desarrollo estándar.

Requerimientos de hardware y software

- Computadora con sistema operativo Windows 10 o superior, o alguna distribución Linux actual.
- Compilador C++ con soporte para el estándar C++17 (por ejemplo, g++ 9+ o MinGW-w64 en Windows).
- Espacio en disco suficiente para almacenar el proyecto y los archivos de recetas generados (del orden de algunos MB).
- Editor de texto o IDE (Visual Studio Code, Code::Blocks, Visual Studio, etc.) para revisar y modificar el código fuente si es necesario.

Estructura de archivos del proyecto

include/

Contiene los archivos de cabecera (.hpp) del proyecto, entre ellos:

- category.hpp
- constants.hpp
- ingredient.hpp
- recipe.hpp
- recipe_list.hpp
- singly_linked_list.hpp
- doubly_linked_list.hpp
- menu.hpp
- storage.hpp

src/

Contiene los archivos de implementación (.cpp), por ejemplo:

- main.cpp
- menu.cpp
- recipe.cpp
- storage.cpp

Compilación en línea de comandos (g++)

En caso de utilizar g++ desde la terminal, se puede compilar el proyecto con un comando similar al siguiente, ejecutado desde la carpeta raíz del proyecto:

```
g++ -std=c++17 -Wall -Wextra -Iinclude src/main.cpp src/menu.cpp  
src/recipe.cpp src/storage.cpp -o cookbook
```

En Windows, el ejecutable se generará como cookbook.exe.
En Linux, se generará como cookbook.

Si se utiliza un IDE, se debe crear un proyecto nuevo, agregar todos los archivos .cpp al proyecto, configurar la carpeta include/ como ruta de cabeceras adicionales, y compilar con estándar C++17.

Ubicación de archivos de datos

Los archivos de recetario que genera el sistema (por ejemplo, recetario.txt) se crean en la ruta que indique el usuario cuando utiliza la opción de “Guardar a disco”.

Manual de operación

El manual de operación describe cómo utilizar el sistema una vez que ha sido compilado correctamente.

Inicio del programa

Para ejecutar el sistema, se debe abrir una terminal en la carpeta donde se encuentra el ejecutable y ejecutar:

- En Windows:
cookbook.exe
- En Linux:
./cookbook

Al iniciar, el programa muestra un menú principal con las distintas opciones numeradas para gestionar el recetario (listar, agregar, buscar, modificar, eliminar, guardar y cargar).

Flujo general de uso

Un flujo típico de uso del sistema es el siguiente:

1. Crear o cargar recetario
 - Si no existe un archivo previo, el usuario puede comenzar agregando recetas con la opción “Agregar receta”-
 - Si ya existe un recetario guardado, se puede utilizar la opción “Leer de disco” para cargarlo en memoria antes de trabajar.
2. Agregar recetas
 - Seleccionar la opción “Agregar receta”.
 - Especificar nombre del platillo, autor, categoría (Desayuno, Comida, Cena o Navideño), tiempo de preparación en minutos y procedimiento.
 - El sistema permite también capturar ingredientes: por cada ingrediente se solicita nombre, cantidad numérica y unidad.
 - Los ingredientes se insertan de forma ordenada alfabéticamente por nombre.

3. Consultar recetas

- Con la opción “Listar recetas” se muestran todas las recetas almacenadas, con su categoría, autor, tiempo, procedimiento e ingredientes.
- Con la opción “Listar por categoría” el usuario puede filtrar únicamente las recetas de una categoría específica.
- Con la opción de “Buscar receta” se puede localizar una receta directamente por nombre.

4. Modificar recetas existentes

- Es posible agregar ingredientes adicionales a una receta ya creada, respetando el orden alfabético de la lista de ingredientes.
- También se puede eliminar un ingrediente específico, modificar la cantidad de un ingrediente o cambiar el procedimiento completo de una receta.
- Estas operaciones se realizan seleccionando primero la receta por nombre, y luego el ingrediente o campo a modificar.

5. Eliminar recetas

- La opción de “Eliminar receta” permite borrar una receta específica del recetario utilizando su nombre.
- La opción de “Eliminar TODAS las recetas” vacía completamente la lista doblemente ligada; después de usarla, el recetario queda vacío y se puede volver a poblar desde cero o cargar desde disco.

6. Ordenar recetas

- El usuario puede ordenar la lista de recetas por nombre o por tiempo de preparación.
- Después de aplicar el ordenamiento recursivo, las recetas se muestran ordenadas al listar el recetario.

7. Guardar y cargar información

- La opción “Guardar a disco” permite guardar todo el recetario (incluyendo ingredientes) en un archivo de texto.
- La opción “Leer de disco” permite reconstruir en memoria el recetario a partir de un archivo guardado previamente.
- Esto permite continuar el trabajo en diferentes sesiones sin perder la información.

8. Finalizar el programa

- Para salir del sistema se utiliza la opción de “Salir” del menú principal.
- Es recomendable guardar el recetario antes de salir si se han realizado cambios.

Mensajes y manejo de errores

El sistema muestra mensajes claros cuando:

- Una receta no existe al intentar buscarla, modificarla o eliminarla.
- Un ingrediente no se encuentra al intentar modificarlo o eliminarlo.
- La lista de recetas está vacía al intentar listar u ordenar.
- El archivo de texto no puede abrirse al guardar o al leer desde disco.

En todos estos casos, el programa informa al usuario la causa del problema y regresa de forma segura al menú principal sin cerrarse inesperadamente.

